DIVIDE AND CONQUER - SORTING ALGORITHMS

MERGE SORT AND QUICK SORT

```
import time
import sys

sys.setrecursionlimit(10000)
```

1. MERGE SORT WITH COMPARISON COUNT

```
def merge_sort_count(arr):
    comparisons = [0]

    def merge_sort(a):
        if len(a) <= 1:
            return a

        mid = len(a) // 2

        left = merge_sort(a[:mid])
        right = merge_sort(a[mid:])

        result = []
        i = 0
        j = 0

        while i < len(left) and j < len(right):
            comparisons[0] += 1

            if left[i] <= right[j]:
                result.append(left[i])
                i += 1
            else:
                result.append(right[j])
                j += 1

        result.extend(left[i:])
        result.extend(right[j:])

        return result

    return merge_sort(arr), comparisons[0]


print("\n========== Q1: MERGE SORT ==========")

arr = [12, 4, 78, 23, 45, 67, 89, 1]
sorted_arr, comparisons = merge_sort_count(arr)

print("Sorted Array:", sorted_arr)
print("Comparisons:", comparisons)
```

```
arr = [5, 2, 9, 1, 6, 3]
sorted_arr, comparisons = merge_sort_count(arr)

print("\nSecond Test Case")
print("Sorted Array:", sorted_arr)
print("Comparisons:", comparisons)
```

```
========== Q1: MERGE SORT ==========
Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16

Second Test Case
Sorted Array: [1, 2, 3, 5, 6, 9]
Comparisons: 10
```

2. QUICK SORT WITH COMPARISON COUNT (Last element as pivot)

```
def quick_sort_count(arr):
    a = arr.copy()
    comparisons = [0]

    def partition(low, high):
        pivot = a[high]
        i = low - 1

        for j in range(low, high):
            comparisons[0] += 1

            if a[j] <= pivot:
                i += 1
                a[i], a[j] = a[j], a[i]

        a[i + 1], a[high] = a[high], a[i + 1]

        return i + 1

    def quick_sort(low, high):
        if low < high:
            p = partition(low, high)
            quick_sort(low, p - 1)
            quick_sort(p + 1, high)

    quick_sort(0, len(a) - 1)

    return a, comparisons[0]


print("\n========== Q2: QUICK SORT ==========")

arr = [38, 27, 43, 3, 9, 82, 10]
sorted_arr, comparisons = quick_sort_count(arr)

print("Sorted Array:", sorted_arr)
print("Comparisons:", comparisons)

arr = [10, 7, 8, 9, 1]
```

```
sorted_arr, comparisons = quick_sort_count(arr)

print("\nSecond Test Case")
print("Sorted Array:", sorted_arr)
print("Comparisons:", comparisons)
```

```
========== Q2: QUICK SORT ==========
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
Comparisons: 11

Second Test Case
Sorted Array: [1, 7, 8, 9, 10]
Comparisons: 10
```

3. MERGE SORT VS QUICK SORT

```
print("\n========== Q3: MERGE SORT VS QUICK SORT ==========")

arr = [12, 11, 13, 5, 6, 7]

merge_result, merge_comp = merge_sort_count(arr)
quick_result, quick_comp = quick_sort_count(arr)

print("Sorted Array:", merge_result)
print("Merge Comparisons:", merge_comp)
print("Quick Comparisons:", quick_comp)

if merge_comp < quick_comp:
    print("Better for this input: Merge Sort")
elif quick_comp < merge_comp:
    print("Better for this input: Quick Sort")
else:
    print("Both have the same comparison count")


print("\nSecond Test Case")

arr = [4, 2, 6, 1, 3]

merge_result, merge_comp = merge_sort_count(arr)
quick_result, quick_comp = quick_sort_count(arr)

print("Sorted Array:", merge_result)
print("Merge Comparisons:", merge_comp)
print("Quick Comparisons:", quick_comp)

if merge_comp < quick_comp:
    print("Better for this input: Merge Sort")
elif quick_comp < merge_comp:
    print("Better for this input: Quick Sort")
else:
    print("Both have the same comparison count")
```

```
========== Q3: MERGE SORT VS QUICK SORT ==========
Sorted Array: [5, 6, 7, 11, 12, 13]
Merge Comparisons: 8
```

```
Quick Comparisons: 9
Better for this input: Merge Sort

Second Test Case
Sorted Array: [1, 2, 3, 4, 6]
Merge Comparisons: 8
Quick Comparisons: 6
Better for this input: Quick Sort
```

4. MERGE SORT EXECUTION TIME

```
def merge_sort_only(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort_only(arr[:mid])
    right = merge_sort_only(arr[mid:])

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):

        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result


print("\n========== Q4: MERGE SORT EXECUTION TIME ==========")

arr = [9, 8, 7, 6, 5, 4, 3]

start = time.perf_counter()
result = merge_sort_only(arr)
end = time.perf_counter()

print("Sorted Array:", result)
print("Time Taken:", end - start, "seconds")


arr = [1, 2, 3, 4, 5, 6]

start = time.perf_counter()
result = merge_sort_only(arr)
end = time.perf_counter()

print("\nSecond Test Case")
```

```
print("Sorted Array:", result)
print("Time Taken:", end - start, "seconds")
```

```
========== Q4: MERGE SORT EXECUTION TIME ==========
Sorted Array: [3, 4, 5, 6, 7, 8, 9]
Time Taken: 5.4409000028954324e-05 seconds

Second Test Case
Sorted Array: [1, 2, 3, 4, 5, 6]
Time Taken: 4.2860999997174076e-05 seconds
```

5. BEST AND WORST CASE QUICK SORT

```
print("\n========== Q5: QUICK SORT BEST/WORST CASE ==========")

best_case = [1, 2, 3, 4, 5]

result, comparisons = quick_sort_count(best_case)

print("Best Case")
print("Input:", best_case)
print("Sorted:", result)
print("Comparisons:", comparisons)


worst_case = [5, 4, 3, 2, 1]

result, comparisons = quick_sort_count(worst_case)

print("\nWorst Case")
print("Input:", worst_case)
print("Sorted:", result)
print("Comparisons:", comparisons)
```

```
========== Q5: QUICK SORT BEST/WORST CASE ==========
Best Case
Input: [1, 2, 3, 4, 5]
Sorted: [1, 2, 3, 4, 5]
Comparisons: 10

Worst Case
Input: [5, 4, 3, 2, 1]
Sorted: [1, 2, 3, 4, 5]
Comparisons: 10
```

6. HYBRID MERGE SORT WITH INSERTION SORT

```
def insertion_sort(arr):
    a = arr.copy()

    for i in range(1, len(a)):
        key = a[i]
        j = i - 1

        while j >= 0 and a[j] > key:
```

```
            a[j + 1] = a[j]
            j -= 1

        a[j + 1] = key

    return a


def hybrid_merge_sort(arr, threshold=4):

    def hybrid(a):

        if len(a) <= 1:
            return a

        if len(a) <= threshold:
            return insertion_sort(a)

        mid = len(a) // 2

        left = hybrid(a[:mid])
        right = hybrid(a[mid:])

        result = []
        i = 0
        j = 0

        while i < len(left) and j < len(right):

            if left[i] <= right[j]:
                result.append(left[i])
                i += 1
            else:
                result.append(right[j])
                j += 1

        result.extend(left[i:])
        result.extend(right[j:])

        return result

    return hybrid(arr)


print("\n========== Q6: HYBRID MERGE SORT ==========")

arr = [12, 11, 13, 5, 6, 7, 3, 2]

result = hybrid_merge_sort(arr, threshold=4)

print("Sorted Array:", result)

standard, _ = merge_sort_count(arr)

print("Standard Merge Sort:", standard)


arr = [9, 4, 6, 2, 8, 1]
```

```
result = hybrid_merge_sort(arr, threshold=4)

print("\nSecond Test Case")
print("Sorted Array:", result)
```

```
========== Q6: HYBRID MERGE SORT ==========
Sorted Array: [2, 3, 5, 6, 7, 11, 12, 13]
Standard Merge Sort: [2, 3, 5, 6, 7, 11, 12, 13]

Second Test Case
Sorted Array: [1, 2, 4, 6, 8, 9]
```

7. COUNTING INVERSIONS USING MERGE SORT

```
def count_inversions(arr):

    def merge_sort(a):

        if len(a) <= 1:
            return a, 0

        mid = len(a) // 2

        left, inv_left = merge_sort(a[:mid])
        right, inv_right = merge_sort(a[mid:])

        merged = []
        i = 0
        j = 0
        inversions = inv_left + inv_right

        while i < len(left) and j < len(right):

            if left[i] <= right[j]:
                merged.append(left[i])
                i += 1

            else:
                merged.append(right[j])

                inversions += len(left) - i

                j += 1

        merged.extend(left[i:])
        merged.extend(right[j:])

        return merged, inversions

    return merge_sort(arr)


print("\n========== Q7: COUNTING INVERSIONS ==========")

arr = [2, 4, 1, 3, 5]
```

```
sorted_arr, inversions = count_inversions(arr)

print("Sorted:", sorted_arr)
print("Inversions:", inversions)


arr = [4, 3, 2, 1]

sorted_arr, inversions = count_inversions(arr)

print("\nSecond Test Case")
print("Sorted:", sorted_arr)
print("Inversions:", inversions)
```

```
========== Q7: COUNTING INVERSIONS ==========
Sorted: [1, 2, 3, 4, 5]
Inversions: 3

Second Test Case
Sorted: [1, 2, 3, 4]
Inversions: 6
```

8. QUICK SORT RECURSION DEPTH ANALYSIS

```
def quick_sort_depth(arr):

    a = arr.copy()
    max_depth = [0]

    def partition(low, high):

        pivot = a[high]
        i = low - 1

        for j in range(low, high):

            if a[j] <= pivot:
                i += 1
                a[i], a[j] = a[j], a[i]

        a[i + 1], a[high] = a[high], a[i + 1]

        return i + 1

    def quick_sort(low, high, depth):

        if low < high:

            max_depth[0] = max(max_depth[0], depth)

            p = partition(low, high)

            quick_sort(low, p - 1, depth + 1)
            quick_sort(p + 1, high, depth + 1)

    if len(a) > 0:
        quick_sort(0, len(a) - 1, 1)
```

```
    return a, max_depth[0]


print("\n========== Q8: QUICK SORT RECURSION DEPTH ==========")

arr = [10, 7, 8, 9, 1, 5]

result, depth = quick_sort_depth(arr)

print("Sorted Array:", result)
print("Max Depth:", depth)


arr = [1, 2, 3, 4, 5]

result, depth = quick_sort_depth(arr)

print("\nSecond Test Case")
print("Sorted Array:", result)
print("Max Depth:", depth)
```

```
========== Q8: QUICK SORT RECURSION DEPTH ==========
Sorted Array: [1, 5, 7, 8, 9, 10]
Max Depth: 4

Second Test Case
Sorted Array: [1, 2, 3, 4, 5]
Max Depth: 4
```

9. EXTERNAL MERGE PROCESS SIMULATION

```
def external_merge(a, b):

    i = 0
    j = 0
    result = []

    while i < len(a) and j < len(b):

        if a[i] <= b[j]:
            result.append(a[i])
            i += 1
        else:
            result.append(b[j])
            j += 1

    while i < len(a):
        result.append(a[i])
        i += 1

    while j < len(b):
        result.append(b[j])
        j += 1

    return result
```

```
print("\n========== Q9: EXTERNAL MERGE ==========")

a = [1, 3, 5]
b = [2, 4, 6]

print("Merged Array:", external_merge(a, b))


a = [10, 20]
b = [5, 15, 25]

print("\nSecond Test Case")
print("Merged Array:", external_merge(a, b))
```

```
========== Q9: EXTERNAL MERGE ==========
Merged Array: [1, 2, 3, 4, 5, 6]

Second Test Case
Merged Array: [5, 10, 15, 20, 25]
```

10. SPACE COMPLEXITY COMPARISON

```
def merge_sort_space(arr):

    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort_space(arr[:mid])
    right = merge_sort_space(arr[mid:])

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):

        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result


print("\n========== Q10: SPACE COMPARISON ==========")

arr = [5, 3, 8, 4, 2]
```

```
merge_result = merge_sort_space(arr)
quick_result, _ = quick_sort_count(arr)

print("Sorted:", merge_result)

# Approximate theoretical auxiliary space
n = len(arr)

print("Merge Sort Auxiliary Space: O(n)")
print("Quick Sort Auxiliary Space: O(log n) average")
print("Quick Sort Worst-case Space: O(n)")


arr = [9, 7, 5, 3, 1, 2]

merge_result = merge_sort_space(arr)
quick_result, _ = quick_sort_count(arr)

print("\nSecond Test Case")
print("Sorted:", merge_result)
print("Merge Sort Auxiliary Space: O(n)")
print("Quick Sort Auxiliary Space: O(log n) average")
```

```
========== Q10: SPACE COMPARISON ==========
Sorted: [2, 3, 4, 5, 8]
Merge Sort Auxiliary Space: O(n)
Quick Sort Auxiliary Space: O(log n) average
Quick Sort Worst-case Space: O(n)

Second Test Case
Sorted: [1, 2, 3, 5, 7, 9]
Merge Sort Auxiliary Space: O(n)
Quick Sort Auxiliary Space: O(log n) average
```

11. K-th SMALLEST ELEMENT USING QUICK SORT PARTITION

```
def kth_smallest(arr, k):

    a = arr.copy()

    def partition(low, high):

        pivot = a[high]
        i = low - 1

        for j in range(low, high):

            if a[j] <= pivot:
                i += 1
                a[i], a[j] = a[j], a[i]

        a[i + 1], a[high] = a[high], a[i + 1]

        return i + 1

    def quick_select(low, high, k_index):
```

```
        if low == high:
            return a[low]

        pivot_index = partition(low, high)

        if pivot_index == k_index:
            return a[pivot_index]

        elif k_index < pivot_index:
            return quick_select(
                low,
                pivot_index - 1,
                k_index
            )

        else:
            return quick_select(
                pivot_index + 1,
                high,
                k_index
            )

    return quick_select(0, len(a) - 1, k - 1)


print("\n========== Q11: K-th SMALLEST ==========")

arr = [7, 10, 4, 3, 20, 15]
k = 3

print("K-th Smallest Element:", kth_smallest(arr, k))


arr = [12, 3, 5, 7, 19]
k = 2

print("\nSecond Test Case")
print("K-th Smallest Element:", kth_smallest(arr, k))
```

```
========== Q11: K-th SMALLEST ==========
K-th Smallest Element: 7

Second Test Case
K-th Smallest Element: 5
```

12. PERFORMANCE DATA COLLECTION

```
def performance_data(arr):

    _, merge_comp = merge_sort_count(arr)
    _, quick_comp = quick_sort_count(arr)

    return merge_comp, quick_comp


print("\n========== Q12: PERFORMANCE DATA ==========")
```

```
test_arrays = [
    [5, 4, 3, 2, 1],
    [8, 7, 6, 5, 4, 3, 2, 1],
    [1, 3, 5, 7, 2, 4, 6, 8],
    [10, 2, 8, 4, 6, 1, 9, 3, 7, 5]
]

print("\nN\tMerge Comparisons\tQuick Comparisons")
print("-" * 50)

data = []

for arr in test_arrays:

    merge_comp, quick_comp = performance_data(arr)

    data.append(
        (len(arr), merge_comp, quick_comp)
    )

    print(
        len(arr),
        "\t",
        merge_comp,
        "\t\t\t",
        quick_comp
    )
```

```
========== Q12: PERFORMANCE DATA ==========

N       Merge Comparisons       Quick Comparisons
--------------------------------------------------
5        7                       10
8        12                      28
8        15                      19
10       24                      21
```

EXACT TEST CASES FROM QUESTION 12

```
print("\n========== Q12 TEST CASE 1 ==========")

arr = [5, 4, 3, 2, 1]

merge_result, merge_comp = merge_sort_count(arr)
quick_result, quick_comp = quick_sort_count(arr)

print("Sorted Array:", merge_result)
print("Merge Comparisons:", merge_comp)
print("Quick Comparisons:", quick_comp)


print("\n========== Q12 TEST CASE 2 ==========")

arr = [8, 7, 6, 5, 4, 3, 2, 1]

merge_result, merge_comp = merge_sort_count(arr)
```

```
quick_result, quick_sort_count(arr)

print("Sorted Array:", merge_result)
print("Merge Comparisons:", merge_comp)
print("Quick Comparisons:", quick_comp)
```

```
========== Q12 TEST CASE 1 ==========
Sorted Array: [1, 2, 3, 4, 5]
Merge Comparisons: 7
Quick Comparisons: 10

========== Q12 TEST CASE 2 ==========
Sorted Array: [1, 2, 3, 4, 5, 6, 7, 8]
Merge Comparisons: 12
Quick Comparisons: 10
```

FINAL COMPLEXITY ANALYSIS

Gemini

```
print("\n============================================================")
print("FINAL COMPLEXITY ANALYSIS")
print("============================================================")

print("""
MERGE SORT
Best Case    : O(n log n)
Average Case : O(n log n)
Worst Case   : O(n log n)
Space        : O(n)

QUICK SORT
Best Case    : O(n log n)
Average Case : O(n log n)
Worst Case   : O(n^2)
Space        : O(log n) average
               O(n) worst case

HYBRID MERGE SORT
Time         : O(n log n)
Small parts : Insertion Sort

COUNTING INVERSIONS
Time         : O(n log n)
Space        : O(n)

QUICK SELECT
Average      : O(n)
Worst Case   : O(n^2)

EXTERNAL MERGE
Time         : O(n + m)
Space        : O(n + m)

GENERAL COMPARISON
Merge Sort is predictable and guarantees O(n log n).
Quick Sort is usually fast in practice but depends on
pivot selection and can degrade to O(n^2).
```

```
""")
```

```
============================================================
FINAL COMPLEXITY ANALYSIS
============================================================

MERGE SORT
Best Case    : O(n log n)
Average Case : O(n log n)
Worst Case   : O(n log n)
Space        : O(n)

QUICK SORT
Best Case    : O(n log n)
Average Case : O(n log n)
Worst Case   : O(n^2)
Space        : O(log n) average
               O(n) worst case

HYBRID MERGE SORT
Time         : O(n log n)
Small parts : Insertion Sort

COUNTING INVERSIONS
Time         : O(n log n)
Space        : O(n)

QUICK SELECT
Average      : O(n)
Worst Case   : O(n^2)

EXTERNAL MERGE
Time         : O(n + m)
Space        : O(n + m)

GENERAL COMPARISON
Merge Sort is predictable and guarantees O(n log n).
Quick Sort is usually fast in practice but depends on
pivot selection and can degrade to O(n^2).
```